Two changes to the pipeline

Cut at the bands' inner edges · mend the reference cube

Five captures whose ground truth and whose cameras see the same surface now read within 1% on average.

What changed since the previous commit

cec1b81 → 5ab597f. 134 files and +1996 / −20 lines, but only eight of them are pipeline code.

- Two new modules, each one step inside Stage 3:
  - pipeline/core/band_edges.py — moves the two cutting planes to the bands' inner edges
  - pipeline/core/box_mending.py — makes the reference cube's points describe a cube
- Four switches in pipeline/config.py, both steps on by default:
  - BAND_CUT_EDGE = inner | centre, BOX_MENDING = on | off, BOX_MEND_TOLERANCE_MM = 1.0, BOX_MEND_CELL_MM = 4.0
- Everything else in the commit is measurement, not pipeline:
  - five tape profiles, the run-versus-tape scripts, 19 figures, EXPERIMENT_LOG sections 18-20, and the inputs/test1..test6 rename

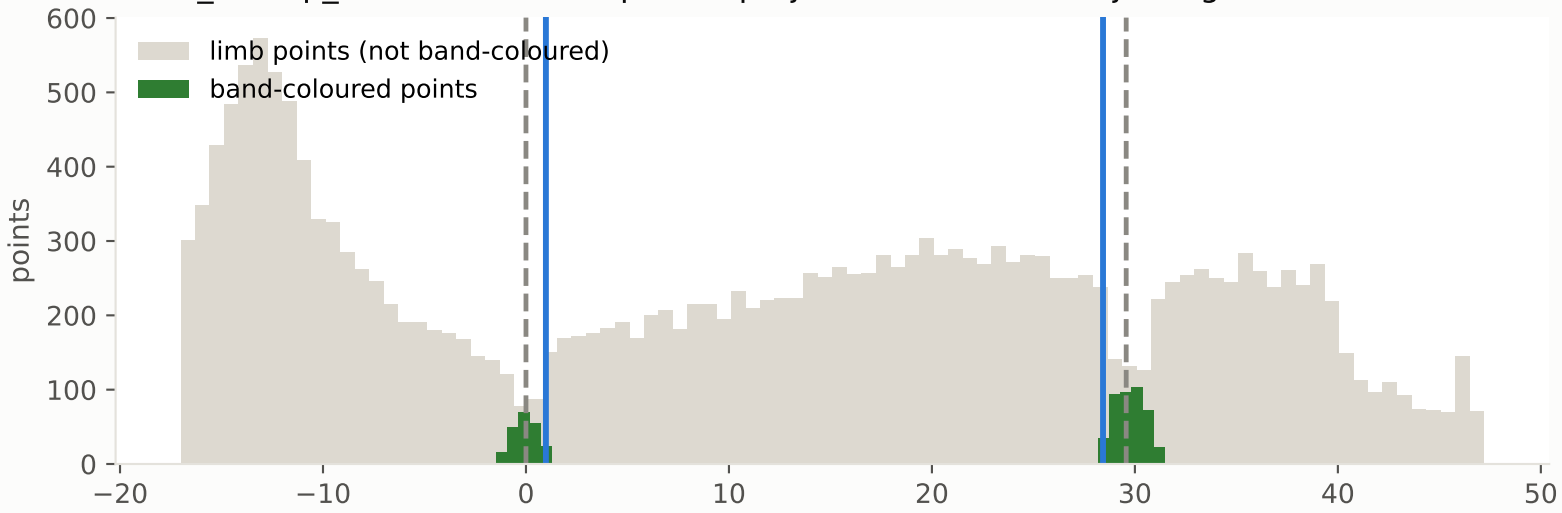
file	lines	what it does
pipeline/core/box_mending.py	+171	flatten the cube's fuzz, fill its gaps
pipeline/core/band_edges.py	+107	slide each cutting plane to its stripe's inner edge
pipeline/stages/clean.py	+40 / −2	calls both, in that order, late in Stage 3
pipeline/config.py	+39 / −6	the four switches, with the measured effect in comments

Source: git diff --stat cec1b81 5ab597f

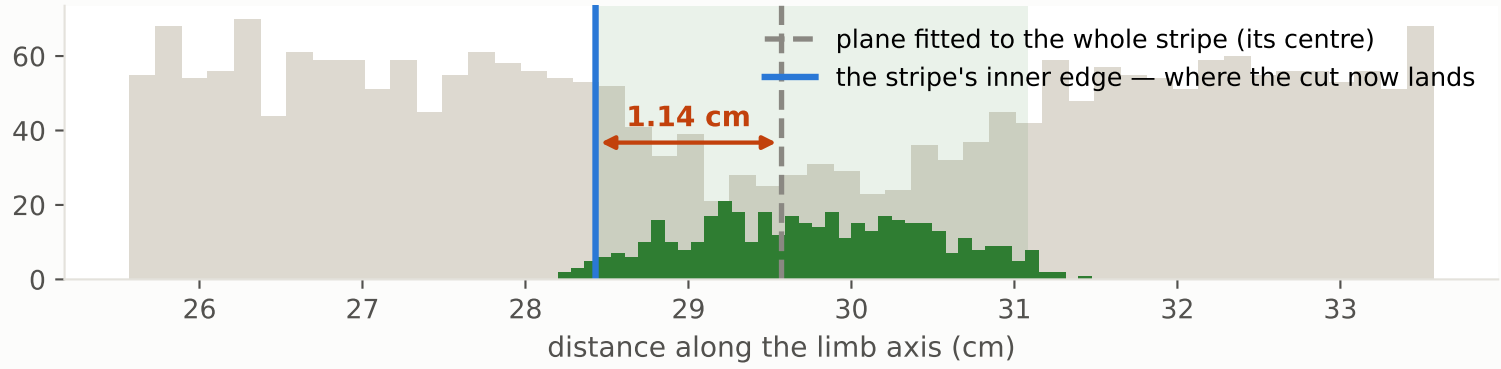
Problem 1: the band is a stripe, not a line

Every volume is measured between two painted bands. A plane fitted to a whole stripe lands in its middle — but the tape is read between the facing edges.

test2_champ_hairless · all limb points, projected onto the line joining the two bands



the upper stripe close up



Measured on this capture

- lower stripe 2.09 cm wide
- upper stripe 2.65 cm wide
- centre to centre 29.57 cm
- inner to inner 27.45 cm
- the tape said 27.0 cm

Across every capture the two choices differ by 2.0–2.5 cm of limb — about 8% of the length, and the single biggest free choice anywhere in the pipeline.

Sliding both planes all the way from the inner edges to the outer edges swings the reported volume by 15–35%.

Source: output_test2_champ_hairless Stage 3 clouds, read with band_edges.py

Fix 1: slide each plane to its stripe's inner edge

The plane keeps the orientation Stage 3 fitted to the band. Only its position moves, along the line joining the two bands.

- Pick out the band-coloured points: green above the mean of red and blue.
- Project them onto the line joining the two band centres.
- Take each stripe's 2nd and 98th percentile as its edges, so colour bleed at the rim is ignored.
- Move the lower plane up to its stripe's top, the upper plane down to its stripe's bottom.
- If either stripe has fewer than 40 points, leave both planes where they were.

What every run now reports

capture	centre to centre	inner to inner	moved
test2_champ_hairless	29.57 cm	27.45 cm	−2.12 cm
test2_sunshine_hairless	29.27 cm	27.02 cm	−2.26 cm
test2_keng	29.15 cm	26.92 cm	−2.24 cm
test1_6	29.19 cm	26.70 cm	−2.49 cm
test1_5	27.96 cm	25.71 cm	−2.25 cm

What it was worth

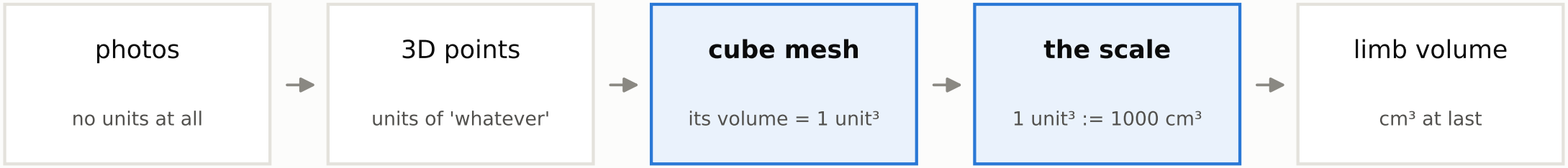
capture	before	after
test1_6	+6.9%	+0.2%
test2_keng	+6.9%	+1.7%
test1_5	+4.8%	+0.1%

Volumes with both changes on. The cut is by far the larger of the two terms; mending moves a sound capture by less than half a percent.

Source: cutting_line_levelled.json from each run; EXPERIMENT_LOG §20

Box mending, from the beginning: what the cube is for

A photograph has no scale. The 10 cm ArUco cube standing in the scene is the only thing in it whose size we know.



Stage 6 takes the scale from the cube MESH's volume, not from its points. So whatever the mesher does to the cube is divided into every number we report: a 1% error in the cube's volume is a 1% error in the limb's.

Why the cube may be corrected and the limb may not

The cube is the one object in the scene whose true shape is known exactly — six flat faces, all 10 cm, all at right angles. Correcting its points invents nothing; it only enforces what we already know.

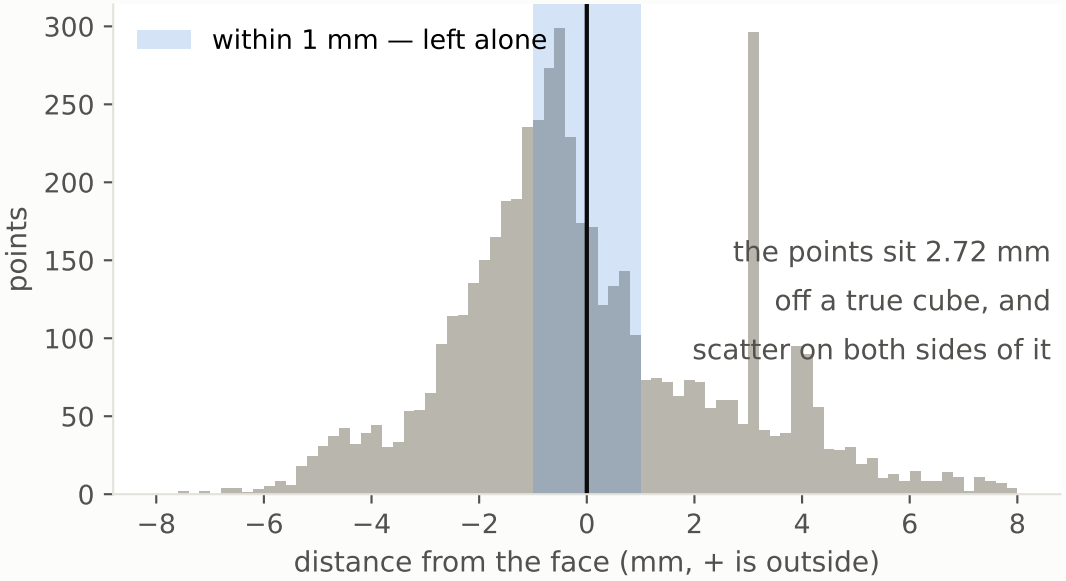
The limb's true shape is what we are trying to measure. Nothing may be enforced on it, so none of this is applied there.

Source: pipeline/stages/volume.py; pipeline/core/box_mending.py docstring

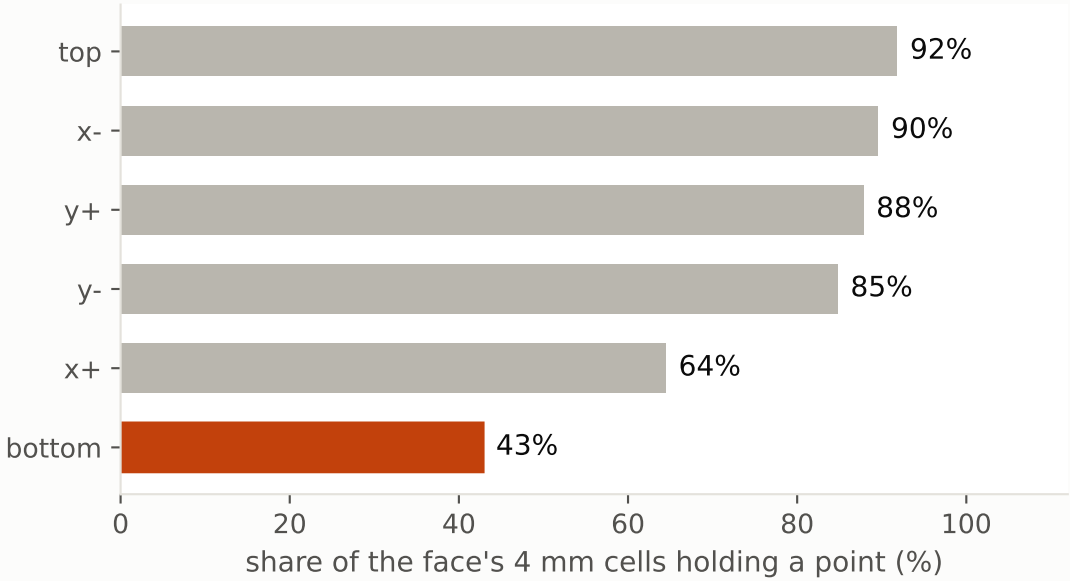
What is wrong with the cube's points: fuzz, and gaps

test2_champ, a capture where the cube happened to be photographed nearly corner-on. 5,418 points.

the fuzz: how far each point sits off its nearest face



the gaps: how much of each face has any points on it



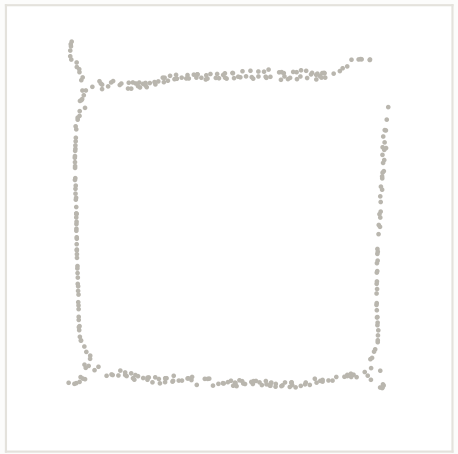
The gaps are the expensive one. A face with holes in it cannot be closed by Poisson reconstruction — the surface tunnels straight through. Stage 4 detects that (Euler characteristic ≠ 2) and falls back to the alpha-shape ladder, whose loosest rung wraps the points from outside: on this capture it floated 2.9 mm clear on every side, 9.1% more volume than the cube its own points describe.

Source: box_mending.py run on output_test2_champ; Stage 4 log, 2026-10-09

How mending works: four steps, nothing invented

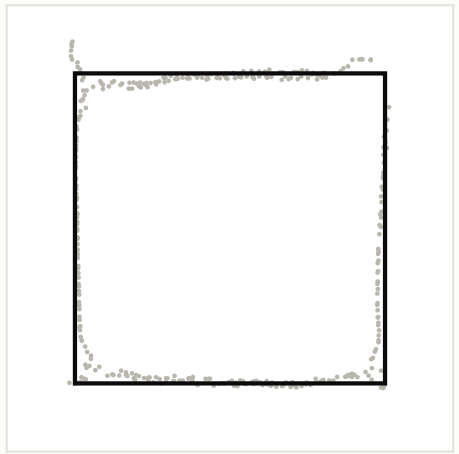
Panels 1–3 are one horizontal 1 cm slab at the cube's mid-height, seen from above; panel 4 looks straight at the bottom face. The cube is never chopped into chunks — that is only how the picture is drawn.

1 · the points as they arrive



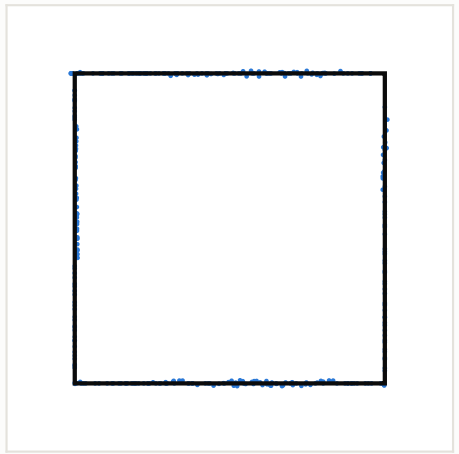
A band of scatter either side of where each face must be, and whole stretches with nothing on them at all.

2 · fit an ideal 10 cm cube



Yaw sweep, then Nelder–Mead on five unknowns: yaw, centre x and y, base height and side. The residual is taken over ALL points, so the fit sits in the middle of the scatter rather than on its outside.

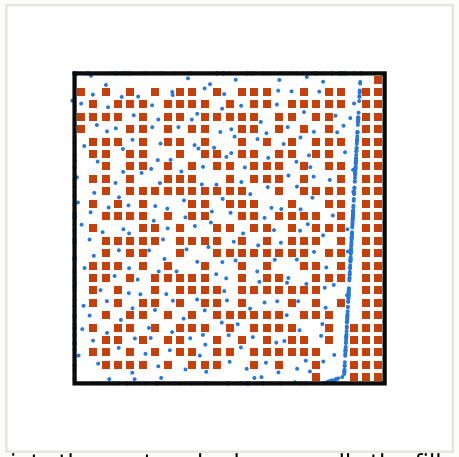
3 · snap the fuzz onto the faces



Every point more than 1 mm off its nearest face is moved onto it.

1,608 pulled in, 1,925 pushed out, 1,885 already close enough.

4 · fill every empty cell



Each face is gridded into 25 × 25 cells of 4 mm, and a point (orange) goes into every empty one.

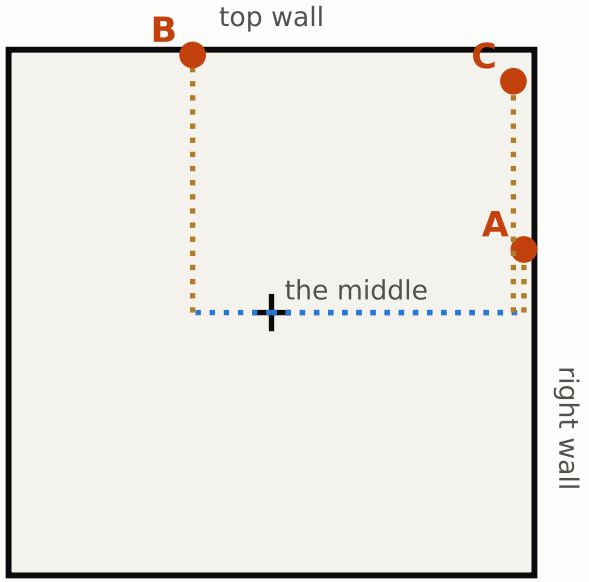
866 added here; this face was only 43% covered. 5,418 → 6,284.

Mending does not decide how big the cube is — the fit is driven by the points, so on this capture 42% of them end up outside the fitted cube and 58% inside, mean offset +0.10 mm. Where the points say the cube reconstructed small, mending makes the mesh say so too, instead of hiding it under a loose wrap.

Source: pipeline/core/box_mending.py run live on output_test2_champ

The same idea, flattened: which wall is this dot on?

A box seen from above is just a square. Every dot is asked two questions, and the bigger answer decides.



How far from the middle, in each direction?

dot A → right wall

4.8 cm sideways

1.2 cm up

clear: 36 mm ahead

dot B → top wall

1.5 cm sideways

4.9 cm up

clear: 34 mm ahead

dot C → right wall

4.6 cm sideways

4.4 cm up

almost a tie — only 2 mm in it

That is the whole rule

Stand in the middle of the box. Measure how far the dot is from you sideways, and how far it is up. Whichever is bigger tells you which wall the dot is nearly touching.

A real box has three directions instead of two: three numbers, six walls. Nothing else changes.

No other dot is consulted, and no edge is traced. Each dot answers on its own.

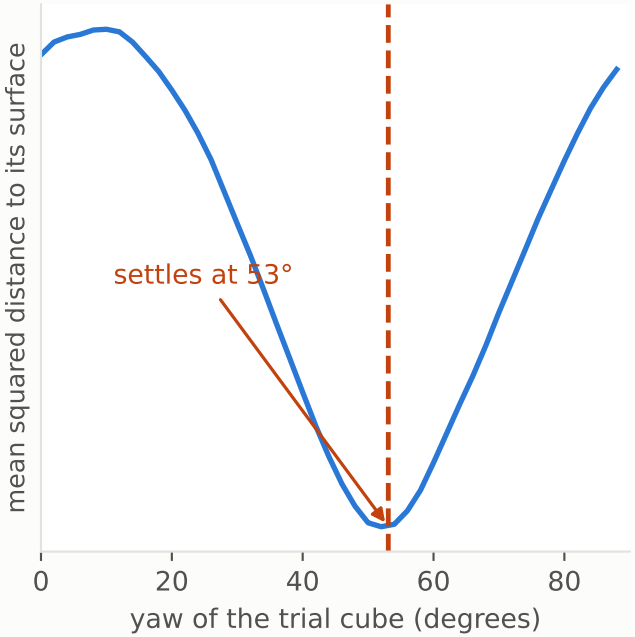
Dot C is the awkward one. It sits in a corner, so it is almost the same distance from two walls and the rule nearly ties. That is fine: a dot in a corner is touching both walls anyway, so either answer puts it in almost the same place. On the real cube 8% of the dots are like C, and the rest win by a wide margin — the next slide measures it.

Source: the rule in pipeline/core/box_mending.py, nearest_face()

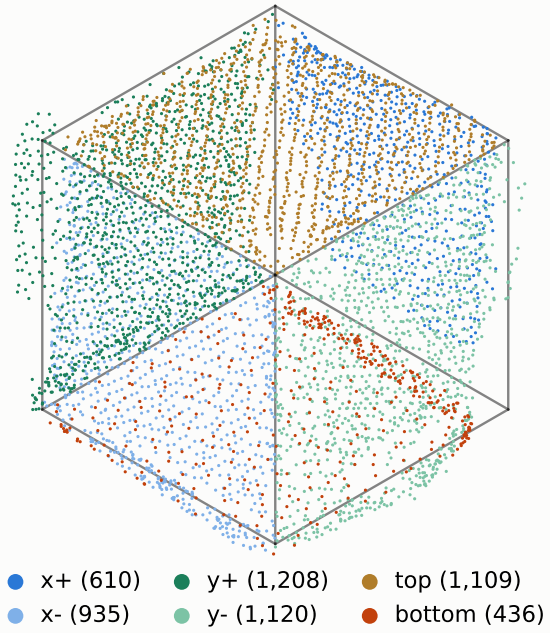
How each point is told which face it is on

A fair question when the cube is seen corner-on. The faces are not worked out from edges or from neighbouring faces: the cube's pose is fitted first, then each point is judged alone.

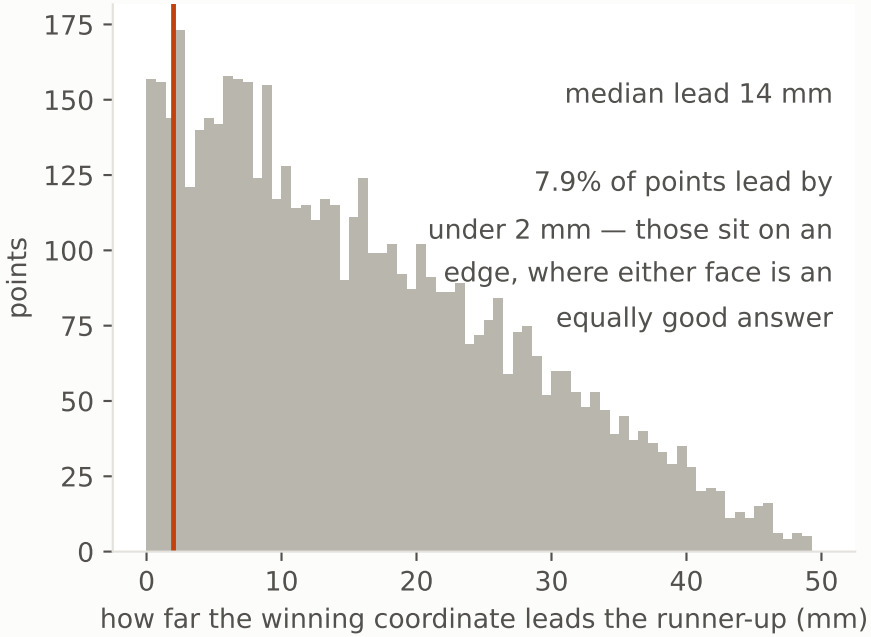
1 · sweep the yaw to find the cube



2 · the largest coordinate names the face



3 · how decisive each assignment is



The rule, in full

rotate the point by minus the fitted yaw, move the cube's centre to the origin, then take whichever of |x|, |y|, |z| is largest: that axis is the face, and its sign is the side.

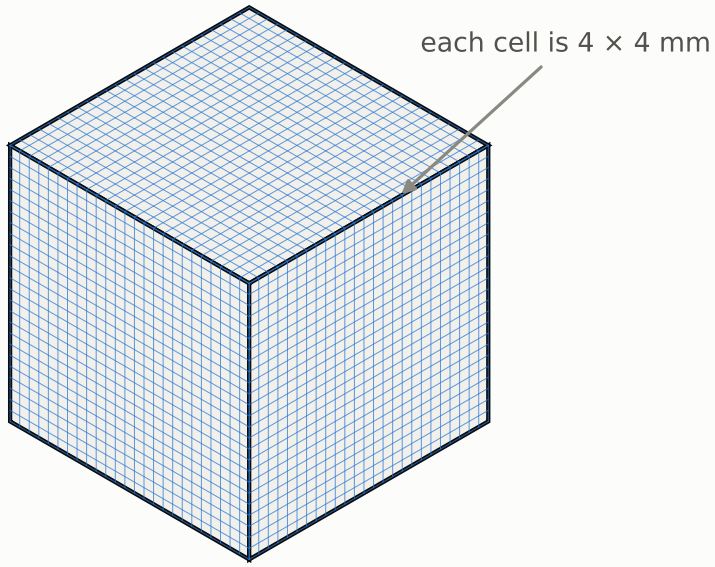
The fit itself solves five unknowns — yaw, centre x and y, base height and side — by minimising every point's distance to the trial cube's surface. The yaw is swept before any local fitting because a cube fitted 45° out would wrap its own diagonal and read about 40% large.

Source: cube_fit.fit_cube and box_mending.nearest_face on output_test2_champ

How the cube is sliced: a 4 mm grid on each of its six faces

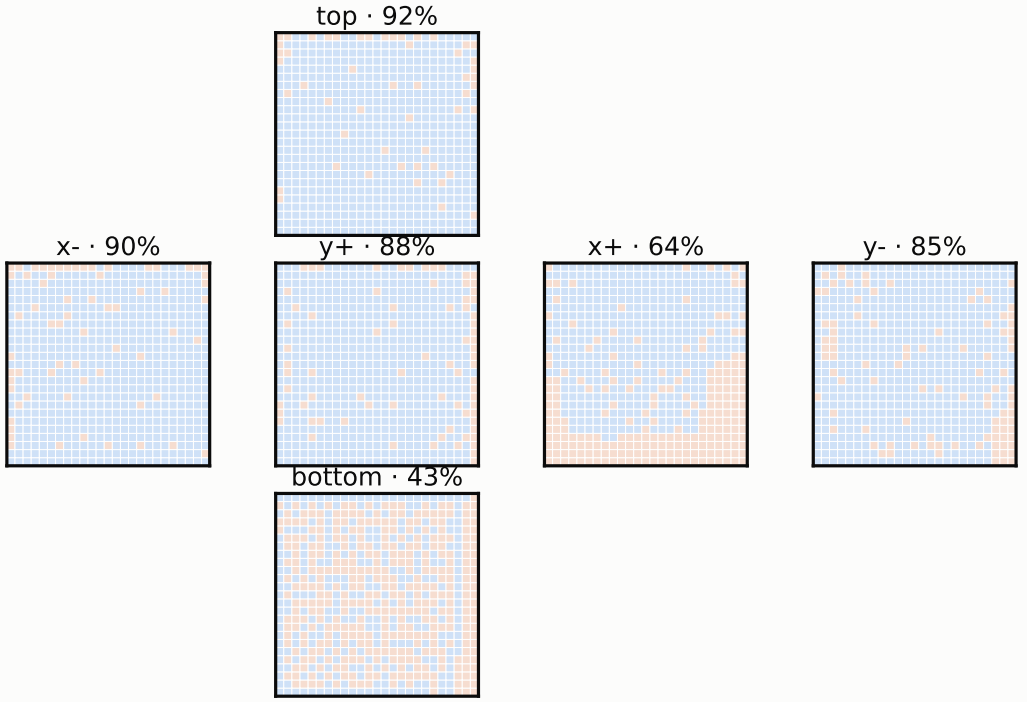
The cube is never cut into blocks or layers. Each face is gridded on its own, and only on the surface.

the 10 cm cube, 25 × 25 cells per face



the same cube unfolded, one square per cell

shaded by what test2_champ's capture actually reached



Reading it

a cell that held a point

an empty cell — the filler drops one point in

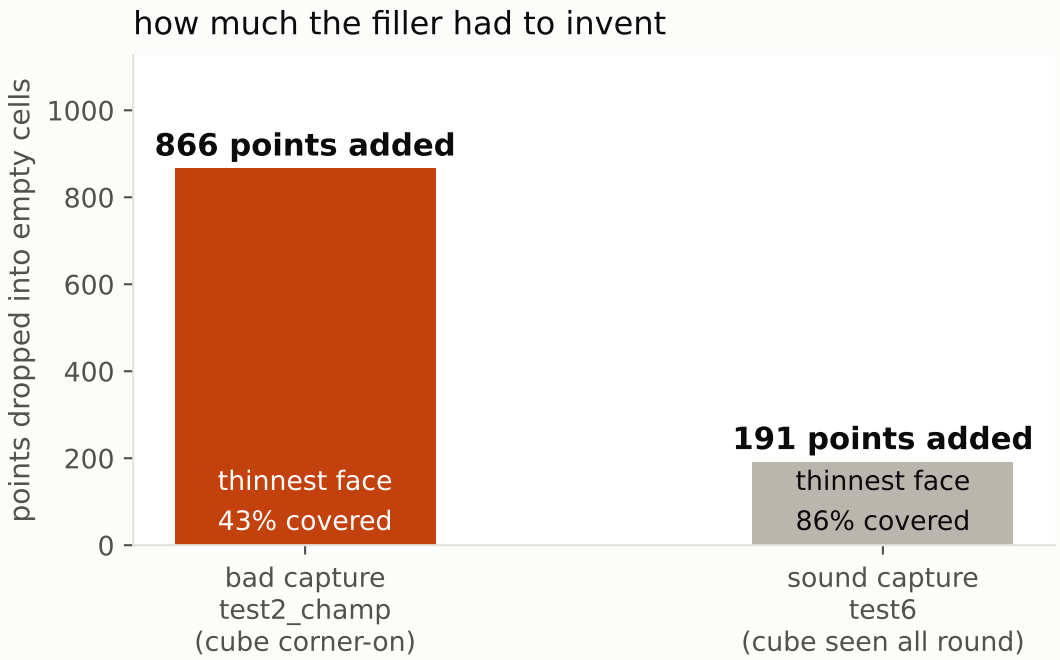
2,884 of the 3,750 cells held a point, so 866 were filled.

Why a grid and not a stack of slices: the cube's error is that parts of its SURFACE were never photographed, and a hole in a surface is what makes Poisson reconstruction tunnel. A flat grid on each face finds exactly those holes; cutting the cube into layers would not say which patch of which face is missing. The bottom face looks at the floor, so it is always the thinnest — 43% here — and it is most of the filler's work.

Source: pipeline/core/box_mending.py run live on output_test2_champ

What mending is worth — and when it does nothing

The step earns its keep on a capture that went wrong, and is close to a no-op on one that went right.



Measured consequences

- Poisson closes on the mended cube instead of tunnelling, so Stage 4 stops falling back to the alpha ladder.
- test2_sunshine, a sound capture: the cube's mesh went 1000.0 → 1004 cm³. Four parts in a thousand.
- test2_champ's alpha wrap had been reading 9.1% more volume than its own points described. That term is gone.
- On a sound capture the step is invisible in the result — which is the point. It is a guard, not a correction.

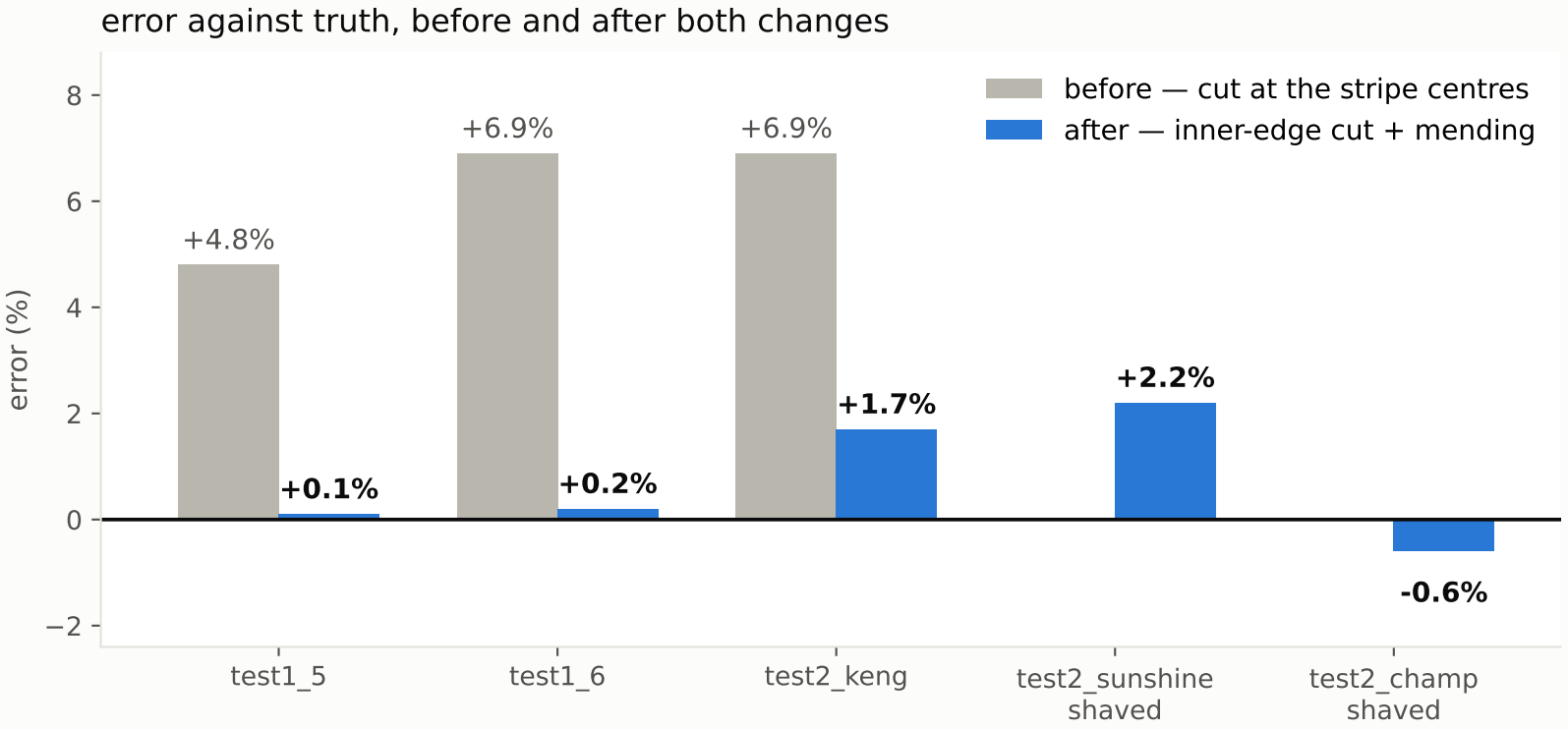
What it is NOT

- Not a way to force the cube to 10 cm — the fit is 10 cm by construction, and the points keep their own spread around it.
- Not a rescue for a barely photographed cube: filling a face that is 43% covered is still mostly guessing. Re-shoot instead.
- Not applied to the limb, whose true shape is the unknown we are after.

Source: box_mending.py on output_test2_champ and output_test6; EXPERIMENT_LOG §20

What the two changes measured

Signed error against each capture's own ground truth. Positive means the pipeline read more volume than the truth did.



The two shaved captures have no 'before': they were only ever shot after the change.

Five captures

1.0%

mean absolute error

+0.7%

mean signed bias — no systematic over-reading left

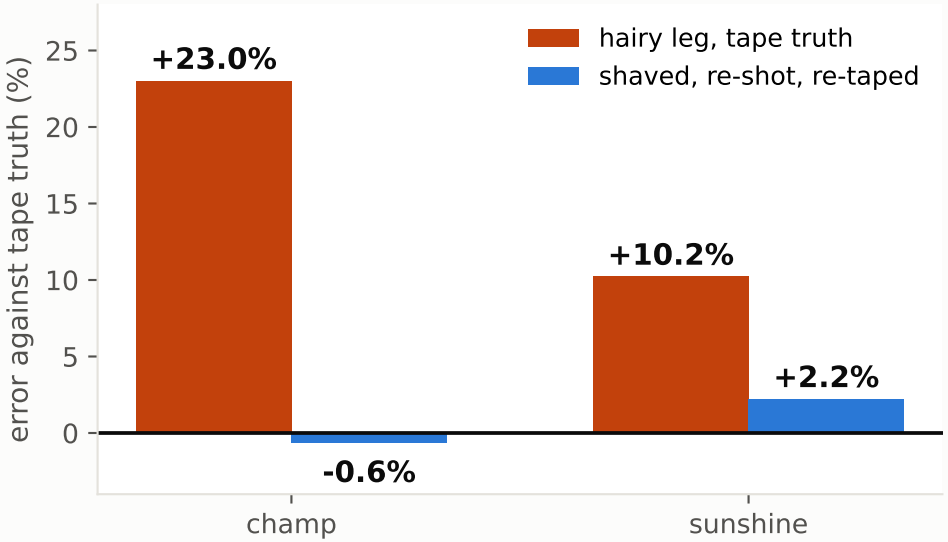
All five are captures where the truth and the cameras see the same surface: a shaved or hairless leg measured with a tape, or a hairy leg measured by water displacement.

Source: EXPERIMENT_LOG §20; run logs of 2026-10-09

The other finding: leg hair, and a lesson about fitting

A tape flattens hair and reads the skin. The cameras see the outside of it. On a hairy leg the two truths are not the same surface.

shaving one leg and capturing it again



What the shave actually measured

region of the limb	girth removed	implied hair
lower half	+0.11 cm	0.02 cm
upper half, where the hair was	+0.49 cm	0.08 cm
peak, 70–95% along	+0.80 to +0.89 cm	0.13–0.14 cm

So VGGT reads 0.8–1.4 mm of apparent radius off hair that stands 4–8 mm: it sees mostly through it, sits a little proud, and only where the hair is.

The lesson

Earlier the same day a hair term was added to the tape truth and solved for. It 'explained' sunshine at 0.41 cm of hair and champ at 0.90 cm — both inside the ranges measured on those legs by hand, which looked like confirmation and was not: one free parameter against one target always fits. The shave is what actually measured it.

A bad capture still costs more than hair does. The first shaved upload read +9.6% and a re-shoot of the same leg against the same tape read +2.2% — 7.3% apart, with nothing about the subject changed in between.

Source: EXPERIMENT_LOG §20; the test2_*_hairless tape profiles

Where this leaves us

Both changes are on by default on keng-branch at 5ab597f, and either can be switched off for a single run.

Settled

- Where the cutting plane lands on a 2 cm stripe was the largest error in the pipeline, and it now has a definition: the inner edges, which is what the tape measures.
- The reference cube never held 6% of error. Its volume was already right to about 0.4% on a sound capture.
- On a capture whose truth and cameras see the same surface, the pipeline reads within about 1%.
- Leg hair is a truth-definition problem, not a model problem. Shave the band region, or measure the truth by water displacement.

Still open

- Four older water-truth captures read −5.9%, +7.1%, −13.1% and −10.8%. Where the water line sat on the band was never written down, so their truth belongs to a different plane than the one we now cut on.
- Repeatability between two captures of the same leg is 7.3%. Capture quality now dominates everything else, and nothing in the pipeline measures it.
- Stage 0 still refuses any frame that does not show the whole cube. Three captures in a row hit that gate from poses where the cube cannot fit in frame; it should warn, not stop, once enough frames carry it.

Source: EXPERIMENT_LOG §21